

COMPE 510 – PA 2 Report

Rhilo Sotto
9/28/2025

Results

Part 1: Normal Equations

Fprintf result of part 1

```
===== Part 1: Normal Equations =====  
Warning: Column headers from the file were modified to  
table. The original column headers are saved in the Va  
Set 'VariableNamingRule' to 'preserve' to use the origi  
First 10 examples from the training dataset:  
x = [38 643 3], y = 32  
x = [10 279 7], y = 52  
x = [15 393 6], y = 8  
x = [4 757 3], y = 37  
x = [10 276 5], y = 34  
x = [12 562 5], y = 42  
x = [4 56 7], y = 54  
x = [32 5512 1], y = 17  
x = [41 402 4], y = 35  
x = [8 640 5], y = 41  
  
beta computed from the normal equations:  
43.628134  
-0.262581  
-0.005628  
1.289454  
  
Program paused. Press enter to continue.
```

Part 2: Evaluate Performance

Fprintf result of part 2

```
===== Part 2: Evaluate Performance =====  
Average prediction error (using normal equations):  
    58.275890  
  
Predicted price of the house (using normal equations):  
    $50.604743
```

Part 3: Feature Normalization

Fprintf result of part 3

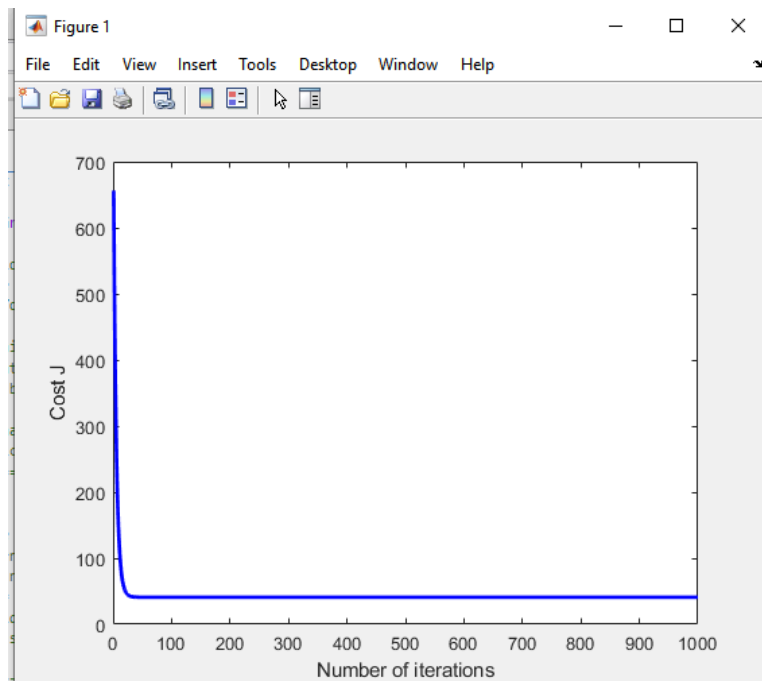
```
===== Part 3: Feature Normalization =====  
Normalizing Features ...  
beta computed from the normal equations after feature normalization:  
    37.748621  
    -3.050995  
    -7.524038  
     3.931880  
  
Average prediction error after normalization (using normal equations):  
    58.275890  
  
Program paused. Press enter to continue.
```

Part 4: Gradient Descent

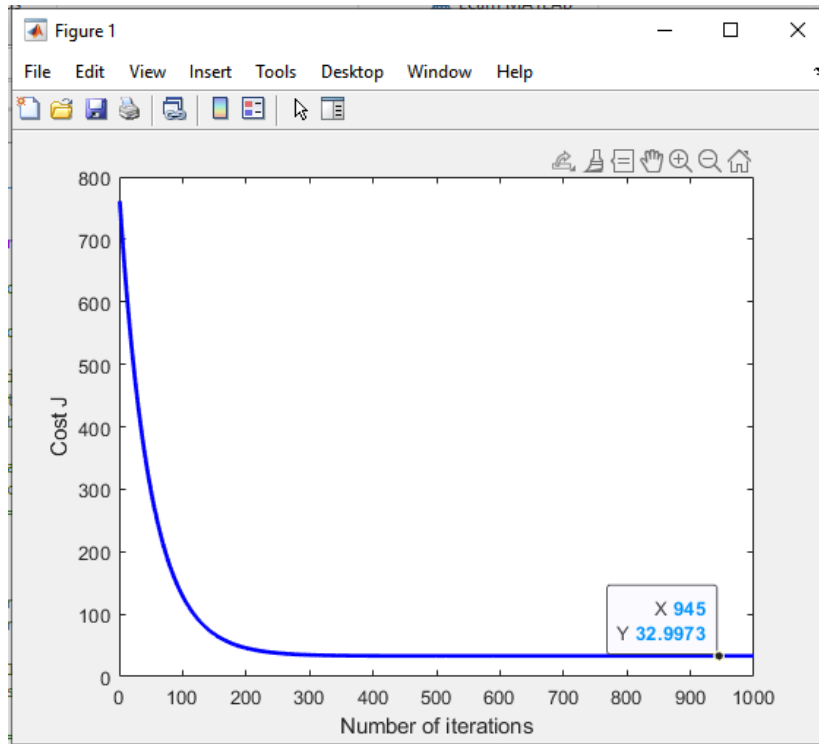
Fprintf result of part 4

```
===== Part 4: Gradient Descent =====  
beta computed from gradient descent:  
37.746991  
-3.054349  
-7.495025  
3.960975  
  
Average prediction error (using gradient descent):  
58.251341  
  
Predicted price of the house (using gradient descent):  
$50.631828
```

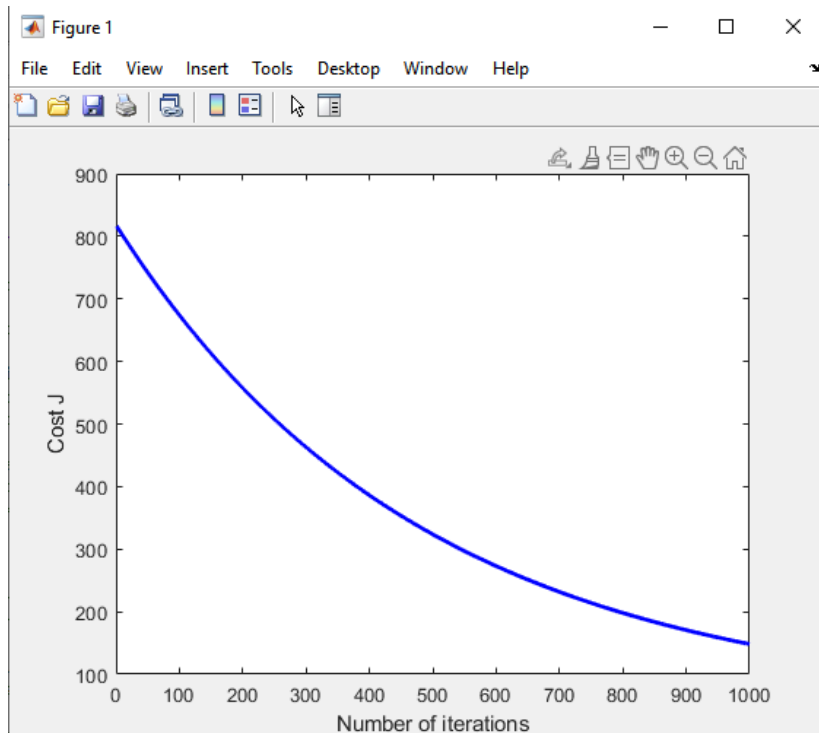
Graph with $\alpha = 0.1$, num_iters = 1000



Graph with $\alpha = 0.01$, num_iters = 1000



Graph with $\alpha = 0.001$, num_iters = 1000



Questions:

1. The cost function for all choices of learning rate (α) managed to show convergence or heading towards convergence. With higher learning rates, the model converged in less iterations, as shown by the slope, compared to lower learning rates, which was still in the midst of converging even after 1000 iterations. The learning rate of 0.1 worked best, managing to converge in less than 100 iterations.
2. Feature normalization takes all the features and sets their scale to be on the same level, with a mean of 0 and standard deviation of 1. Doing this allows gradient descent to move more smoothly toward the minimum, since each step would be proportionally similar. Incorrect normalization would lead to wrong predictions since the scale of the test features would be unexpected for the model.
3. For my results both methods had similar test performance, the average prediction error was usually similar up to two decimal places, with neither having a significantly lower error compared to the other. It seems that both methods reached very similar beta values, so their error would likewise be very similar.
4. With more input features, I would need to load more data and normalize those features, the dimensions would all need to increase to accommodate the new data, and I would need to retrain the model to take into account the new features. The new predictions could be more weighted toward the new input features (maybe higher levels are more costly than lower levels), which could throw off current predictions.

Implementation

Implementing the data loading function went well, there was a very convenient Matlab function for splitting the data into training and testing sets. Remembering to add intercept terms to all the sets was simple but kept causing issues when they were forgotten.

The current implementation could be improved further by using built-in Matlab functions that may be more optimized.